

DEVARA HEPSIBA

2403A51L25

BATCH – 51

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task 1 (Sorting – Merge Sort Implementation)

Prompt: Generate a Python list to store hospital patient records. Each patient should have: - Patient ID - Name - Severity level (1-5) - Admission date - Bill amount Add at least 5 sample records.

Code:

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity: O(n log n) - in all cases (best, average, worst)
    Space Complexity: O(n) - due to temporary arrays used during merging

    Args:
        arr (list): A list of comparable elements to be sorted

    Returns:
        list: A new sorted list in ascending order
    """
    if len(arr) <= 1:
        return arr
    # Divide the array into two halves
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    # Recursively sort both halves
    left = merge_sort(left)
    right = merge_sort(right)

    # Merge the sorted halves
    return merge(left, right)

def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Args:
        left (list): First sorted list
        right (list): Second sorted list

    Returns:
        list: A merged sorted list
    """
    result = []
    i = j = 0
```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

result.extend(left[i:])
result.extend(right[j:])

return result

# Test Cases
if __name__ == "__main__":
    # Test 1: Random unsorted list
    test1 = [38, 27, 43, 3, 9, 82, 10]
    print(f"Test 1 - Random list: {test1}")
    print(f"Sorted: {merge_sort(test1)}\n")

    # Test 2: Already sorted list
    test2 = [1, 2, 3, 4, 5]
    print(f"Test 2 - Already sorted: {test2}")
    print(f"Sorted: {merge_sort(test2)}\n")

    # Test 3: Reverse sorted list
    test3 = [5, 4, 3, 2, 1]
    print(f"Test 3 - Reverse sorted: {test3}")
    print(f"Sorted: {merge_sort(test3)}\n")

    # Test 4: List with duplicates
    test4 = [3, 1, 4, 1, 5, 9, 2, 6, 5]
    print(f"Test 4 - With duplicates: {test4}")
    print(f"Sorted: {merge_sort(test4)}\n")

    # Test 5: Single element
    test5 = [42]
    print(f"Test 5 - Single element: {test5}")
    print(f"Sorted: {merge_sort(test5)}\n")

    # Test 6: Empty list
    test6 = []
    print(f"Test 6 - Empty list: {test6}")
    print(f"Sorted: {merge_sort(test6)}")

```

Output:

```

Test 1 - Random list: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]

Test 2 - Already sorted: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

Test 3 - Reverse sorted: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]

Test 4 - With duplicates: [3, 1, 4, 1, 5, 9, 2, 6, 5]
Sorted: [1, 1, 2, 3, 4, 5, 5, 6, 9]

Test 5 - Single element: [42]
Sorted: [42]

Test 6 - Empty list: []
Sorted: []

```

Explanation: Merge Sort is a divide-and-conquer sorting algorithm that splits the list and merges it in sorted order.

It has a time complexity of $O(n \log n)$.

It is efficient and suitable for large datasets.

Task 2 (Searching – Binary Search with AI Optimization)

Prompt: Generate a Python function `merge_sort(arr, index)` that sorts a list of patient records based on a given column index. Include: - Proper docstring - Time complexity - Space complexity - Clean and readable code

Code:

```
def binary_search(arr, target):
    """
    Finds the index of a target element in a sorted list using Binary Search.
    Time Complexity: O(log n) - in all cases (best, average, worst)
    Space Complexity: O(1) - only uses a constant amount of extra space
    Args:
        arr (list): A sorted list of comparable elements
        target: The element to search for
    Returns:
        int: The index of the target if found, -1 otherwise
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

Binary Search Test Cases
if __name__ == "__main__":
    print("\n--- Binary Search Tests ---\n")
    # Test 1: Target found in middle
    test_arr1 = [1, 3, 5, 7, 9, 11, 13, 15, 17]
    result1 = binary_search(test_arr1, 9)
    print(f"Test 1 - Find 9 in {test_arr1}: Index {result1}")
    # Test 2: Target found at start
    result2 = binary_search(test_arr1, 1)
    print(f"Test 2 - Find 1 in {test_arr1}: Index {result2}")
    # Test 3: Target found at end
    result3 = binary_search(test_arr1, 17)
    print(f"Test 3 - Find 17 in {test_arr1}: Index {result3}")
    # Test 4: Target not found
    result4 = binary_search(test_arr1, 10)
    print(f"Test 4 - Find 10 in {test_arr1}: Index {result4}")
    # Test 5: Single element found
    result5 = binary_search([5], 5)
    print(f"Test 5 - Find 5 in [5]: Index {result5}")
    # Test 6: Single element not found
    result6 = binary_search([5], 3)
    print(f"Test 6 - Find 3 in [5]: Index {result6}")
```

Output:

```
--- Binary Search Tests ---  
  
Test 1 - Find 9 in [1, 3, 5, 7, 9, 11, 13, 15, 17]: Index 4  
Test 2 - Find 1 in [1, 3, 5, 7, 9, 11, 13, 15, 17]: Index 0  
Test 3 - Find 17 in [1, 3, 5, 7, 9, 11, 13, 15, 17]: Index 8  
Test 4 - Find 10 in [1, 3, 5, 7, 9, 11, 13, 15, 17]: Index -1  
Test 5 - Find 5 in [5]: Index 0  
Test 6 - Find 3 in [5]: Index -1
```

Explanation: Binary Search finds an element in a sorted list by dividing the search space into halves. It works only on sorted data. Its time complexity is $O(\log n)$, making it very fast.

Task 3 (Real-Time Application – Inventory Management System)

Prompt: Generate a Python function `binary_search(arr, target)` to search patient records by patient ID. Conditions: - The list is sorted by patient ID - Return the patient record if found - Return None if not found Include best, average, and worst case time complexities in docstring.

Code:

```
# Inventory Management System  
class Product:  
    def __init__(self, product_id, name, price, stock):  
        self.product_id = product_id  
        self.name = name  
        self.price = price  
        self.stock = stock  
  
    def __repr__(self):  
        return f"ProductID:{self.product_id}, {self.name}, ${self.price}, Stock:{self.stock}"  
  
def search_by_id(products, target_id):  
    """Binary Search for product by ID (assumes sorted by ID)"""  
    left, right = 0, len(products) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if products[mid].product_id == target_id:  
            return products[mid]  
        elif products[mid].product_id < target_id:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return None  
  
def search_by_name(products, target_name):  
    """Linear search for product by name (case-insensitive)"""  
    target = target_name.lower()  
    for product in products:  
        if product.name.lower() == target:  
            return product  
    return None  
  
def sort_by_price(products):  
    """Merge sort products by price (ascending)"""  
    if len(products) <= 1:  
        return products  
    mid = len(products) // 2  
    left = sort_by_price(products[:mid])  
    right = sort_by_price(products[mid:])  
    return merge_by_price(left, right)
```

```

def merge_by_price(left, right):
    """Merge helper for sorting by price"""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].price <= right[j].price:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test Inventory System
print("\n--- Inventory Management System ---\n")
inventory = [
    Product(105, "Laptop", 999.99, 15),
    Product(102, "Mouse", 29.99, 150),
    Product(108, "Monitor", 299.99, 45),
    Product(101, "Keyboard", 79.99, 200),
    Product(107, "USB Cable", 9.99, 500)
]

print("Original Inventory:")
for p in inventory:
    print(p)

print("\nSorted by Price:")
sorted_by_price = sort_by_price(inventory)
for p in sorted_by_price:
    print(p)

print("\nSearch by ID (107):", search_by_id(sorted(inventory, key=lambda x: x.product_id), 107))
print("Search by Name (Monitor):", search_by_name(inventory, "Monitor"))

```

Output:

```

--- Inventory Management System ---

Original Inventory:
Product(ID:105, Laptop, $999.99, Stock:15)
Product(ID:102, Mouse, $29.99, Stock:150)
Product(ID:108, Monitor, $299.99, Stock:45)
Product(ID:101, Keyboard, $79.99, Stock:200)
Product(ID:107, USB Cable, $9.99, Stock:500)

Sorted by Price:
Product(ID:107, USB Cable, $9.99, Stock:500)
Product(ID:102, Mouse, $29.99, Stock:150)
Product(ID:101, Keyboard, $79.99, Stock:200)
Product(ID:108, Monitor, $299.99, Stock:45)
Product(ID:105, Laptop, $999.99, Stock:15)

Search by ID (107): Product(ID:107, USB Cable, $9.99, Stock:500)
Search by Name (Monitor): Product(ID:108, Monitor, $299.99, Stock:45)

```

Explanation: This system manages product records and allows searching and sorting.

Binary Search is used for fast product lookup.

Merge Sort is used for efficient sorting by price or quantity.

Task 4: Smart Hospital Patient Management System

Prompt: Generate a Python function to search hospital patient records by name. Requirements: - Case-insensitive comparison - Return patient details if found - Return None if not found

Code:

```
# Smart Hospital Patient Management System
class Patient:
    def __init__(self, patient_id, name, severity, admission_date, bill_amount):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity # 1-5 (1=low, 5=critical)
        self.admission_date = admission_date
        self.bill_amount = bill_amount

    def __repr__(self):
        return f"Patient(ID:{self.patient_id}, {self.name}, Severity:{self.severity}, Bill:${self.bill_amount:.2f})"

def search_patient_by_id(patients, target_id):
    """Binary Search by Patient ID - O(log n). Requires sorted list."""
    left, right = 0, len(patients) - 1
    while left <= right:
        mid = (left + right) // 2
        if patients[mid].patient_id == target_id:
            return patients[mid]
        elif patients[mid].patient_id < target_id:
            left = mid + 1
        else:
            right = mid - 1
    return None

def search_patient_by_name(patients, target_name):
    """Linear Search by Name - O(n). Works on unsorted list."""
    target = target_name.lower()
    for patient in patients:
        if patient.name.lower() == target:
            return patient
    return None

def sort_by_severity(patients):
    """Merge Sort by Severity (descending) - O(n log n)."""
    if len(patients) <= 1:
        return patients
    mid = len(patients) // 2
    left = sort_by_severity(patients[:mid])
    right = sort_by_severity(patients[mid:])
    return merge_severity(left, right)

def merge_severity(left, right):
    """Merge helper for sorting by severity (descending)."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].severity >= right[j].severity:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```

def sort_by_bill(patients):
    """Merge Sort by Bill Amount (descending) - O(n log n)."""
    if len(patients) <= 1:
        return patients
    mid = len(patients) // 2
    left = sort_by_bill(patients[:mid])
    right = sort_by_bill(patients[mid:])
    return merge_bill(left, right)

def merge_bill(left, right):
    """Merge helper for sorting by bill amount (descending)."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].bill_amount >= right[j].bill_amount:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test Hospital System
print("\n--- Smart Hospital Patient Management System ---\n")
patients = [
    Patient(1005, "John Smith", 3, "2024-01-10", 5000.00),
    Patient(1001, "Alice Johnson", 5, "2024-01-05", 15000.00),
    Patient(1008, "Bob Wilson", 2, "2024-01-12", 2000.00),
    Patient(1003, "Carol White", 4, "2024-01-08", 8500.00),
    Patient(1007, "David Brown", 1, "2024-01-11", 1200.00)
]

print("Original Patient Records:")
for p in patients:
    print(p)

print("\nSorted by Severity (Critical First):")
by_severity = sort_by_severity(patients)
for p in by_severity:
    print(p)

print("\nSorted by Bill Amount (Highest First):")
by_bill = sort_by_bill(patients)
for p in by_bill:
    print(p)

print("\nSearch by ID (1003):", search_patient_by_id(sorted(patients, key=lambda x: x.patient_id), 1003))
print("Search by Name (Alice Johnson):", search_patient_by_name(patients, "Alice Johnson"))

```

Output:

```

--- Smart Hospital Patient Management System ---

Original Patient Records:
Patient(ID:1005, John Smith, Severity:3, Bill:$5000.00)
Patient(ID:1001, Alice Johnson, Severity:5, Bill:$15000.00)
Patient(ID:1008, Bob Wilson, Severity:2, Bill:$2000.00)
Patient(ID:1003, Carol White, Severity:4, Bill:$8500.00)
Patient(ID:1007, David Brown, Severity:1, Bill:$1200.00)

Sorted by Severity (Critical First):
Patient(ID:1001, Alice Johnson, Severity:5, Bill:$15000.00)
Patient(ID:1003, Carol White, Severity:4, Bill:$8500.00)
Patient(ID:1005, John Smith, Severity:3, Bill:$5000.00)
Patient(ID:1008, Bob Wilson, Severity:2, Bill:$2000.00)
Patient(ID:1007, David Brown, Severity:1, Bill:$1200.00)

Sorted by Bill Amount (Highest First):
Patient(ID:1001, Alice Johnson, Severity:5, Bill:$15000.00)
Patient(ID:1003, Carol White, Severity:4, Bill:$8500.00)
Patient(ID:1005, John Smith, Severity:3, Bill:$5000.00)
Patient(ID:1008, Bob Wilson, Severity:2, Bill:$2000.00)
Patient(ID:1007, David Brown, Severity:1, Bill:$1200.00)

Search by ID (1003): Patient(ID:1003, Carol White, Severity:4, Bill:$8500.00)
Search by Name (Alice Johnson): Patient(ID:1001, Alice Johnson, Severity:5, Bill:$15000.00)

```

Explanation: This system stores and manages patient records.
Binary Search helps quickly find patients by ID.
Merge Sort is used to prioritize patients by severity or bill amount.

Task 5: University Examination Result Processing System

Prompt: Create a menu-driven Python program for a Smart Hospital Patient Management System. The system should: 1. Search patient by ID 2. Search patient by Name 3. Sort patients by Severity 4. Sort patients by Bill Amount 5. Exit, Use binary search for ID. Use merge sort for sorting. Use linear search for name.

Code:

```
# University Examination Result Processing System
class ExamResult:
    def __init__(self, roll_number, name, subject, marks):
        self.roll_number = roll_number
        self.name = name
        self.subject = subject
        self.marks = marks

    def __repr__(self):
        return f"Result(Roll:{self.roll_number}, {self.name}, {self.subject}, Marks:{self.marks})"

def search_by_roll_number(results, target_roll):
    """Binary Search by Roll Number - O(log n). Requires sorted list."""
    left, right = 0, len(results) - 1
    while left <= right:
        mid = (left + right) // 2
        if results[mid].roll_number == target_roll:
            return results[mid]
        elif results[mid].roll_number < target_roll:
            left = mid + 1
        else:
            right = mid - 1
    return None

def search_by_name(results, target_name):
    """Linear Search by Name - O(n). Works on unsorted list."""
    target = target_name.lower()
    for result in results:
        if result.name.lower() == target:
            return result
    return None

def sort_by_marks(results):
    """Merge Sort by Marks (descending) - O(n log n)."""
    if len(results) <= 1:
        return results
    mid = len(results) // 2
    left = sort_by_marks(results[:mid])
    right = sort_by_marks(results[mid:])
    return merge_marks(left, right)
```



```

def merge_marks(left, right):
    """Merge helper for sorting by marks (descending)."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].marks >= right[j].marks:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test Examination System
print("\n--- University Examination Result Processing System ---\n")
exam_results = [
    ExamResult(1005, "Alice Kumar", "Mathematics", 92),
    ExamResult(1001, "Bob Singh", "Mathematics", 88),
    ExamResult(1008, "Carol Patel", "Mathematics", 95),
    ExamResult(1003, "David Lee", "Mathematics", 78),
    ExamResult(1007, "Eva Johnson", "Mathematics", 85),
    ExamResult(1002, "Frank Brown", "Mathematics", 90)
]

print("Original Results:")
for r in exam_results:
    print(r)

print("\nSorted by Marks (Highest First - Rank List):")
ranked = sort_by_marks(exam_results)
for rank, r in enumerate(ranked, 1):
    print(f"Rank {rank}: {r}")

print("\nSearch by Roll Number (1005):",
      search_by_roll_number(sorted(exam_results, key=lambda x: x.roll_number), 1005))
print(["Search by Name (Carol Patel):", search_by_name(exam_results, "Carol Patel")])

```

Output:

```

--- University Examination Result Processing System ---

Original Results:
Result(Roll:1005, Alice Kumar, Mathematics, Marks:92)
Result(Roll:1001, Bob Singh, Mathematics, Marks:88)
Result(Roll:1008, Carol Patel, Mathematics, Marks:95)
Result(Roll:1003, David Lee, Mathematics, Marks:78)
Result(Roll:1007, Eva Johnson, Mathematics, Marks:85)
Result(Roll:1002, Frank Brown, Mathematics, Marks:90)

Sorted by Marks (Highest First - Rank List):
Rank 1: Result(Roll:1008, Carol Patel, Mathematics, Marks:95)
Rank 2: Result(Roll:1005, Alice Kumar, Mathematics, Marks:92)
Rank 3: Result(Roll:1002, Frank Brown, Mathematics, Marks:90)
Rank 4: Result(Roll:1001, Bob Singh, Mathematics, Marks:88)
Rank 5: Result(Roll:1007, Eva Johnson, Mathematics, Marks:85)
Rank 6: Result(Roll:1003, David Lee, Mathematics, Marks:78)

Search by Roll Number (1005): Result(Roll:1005, Alice Kumar, Mathematics, Marks:92)
Search by Name (Carol Patel): Result(Roll:1008, Carol Patel, Mathematics, Marks:95)

```

Explanation: This system processes and ranks student results. Binary Search is used to find students by roll number. Sorting algorithms help generate rank lists efficiently.

Task 6: Online Food Delivery Platform

Prompt: Suggest suitable searching and sorting algorithms for a hospital patient management system containing thousands of records. Justify the selection based on: - Efficiency - Dataset size - Time complexity - Real-world suitability Present the answer in table format.

Code:

```
# Online Food Delivery Platform
class Order:
    def __init__(self, order_id, restaurant_name, delivery_time, price, status):
        self.order_id = order_id
        self.restaurant_name = restaurant_name
        self.delivery_time = delivery_time # in minutes
        self.price = price
        self.status = status # "pending", "preparing", "out_for_delivery", "delivered"

    def __repr__(self):
        return f"Order(ID:{self.order_id}, {self.restaurant_name}, Time:{self.delivery_time}min, ${self.price:.2f}, {self.status})"

def search_order_by_id(orders, target_id):
    """Binary Search by Order ID - O(log n). Requires sorted list by ID."""
    left, right = 0, len(orders) - 1
    while left <= right:
        mid = (left + right) // 2
        if orders[mid].order_id == target_id:
            return orders[mid]
        elif orders[mid].order_id < target_id:
            left = mid + 1
        else:
            right = mid - 1
    return None

def search_by_restaurant(orders, target_restaurant):
    """Linear Search by Restaurant Name - O(n). Works on unsorted list."""
    target = target_restaurant.lower()
    for order in orders:
        if order.restaurant_name.lower() == target:
            return order
    return None

def sort_by_delivery_time(orders):
    """Merge Sort by Delivery Time (ascending) - O(n log n)."""
    if len(orders) <= 1:
        return orders
    mid = len(orders) // 2
    left = sort_by_delivery_time(orders[:mid])
    right = sort_by_delivery_time(orders[mid:])
    return merge_delivery_time(left, right)

def merge_delivery_time(left, right):
    """Merge helper for sorting by delivery time (ascending)."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].delivery_time <= right[j].delivery_time:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```

def sort_by_price(orders):
    """Merge Sort by Price (ascending) - O(n log n)."""
    if len(orders) <= 1:
        return orders
    mid = len(orders) // 2
    left = sort_by_price(orders[:mid])
    right = sort_by_price(orders[mid:])
    return merge_price(left, right)

def merge_price(left, right):
    """Merge helper for sorting by price (ascending)."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].price <= right[j].price:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Test Food Delivery System
print("\n--- Online Food Delivery Platform ---\n")
orders = [
    Order(1005, "Pizza Palace", 35, 18.99, "out_for_delivery"),
    Order(1001, "Burger King", 25, 12.50, "preparing"),
    Order(1008, "Sushi Express", 40, 22.99, "pending"),
    Order(1003, "Taco Bell", 20, 15.75, "delivered"),
    Order(1007, "Chinese Wok", 30, 16.99, "out_for_delivery"),
    Order(1002, "Subway", 15, 9.99, "delivered"),
    Order(1006, "Dominos", 28, 25.50, "preparing")
]

print("Original Orders:")
for o in orders:
    print(o)

print("\nSorted by Delivery Time (Fastest First):")
by_time = sort_by_delivery_time(orders)
for o in by_time:
    print(o)

print("\nSorted by Price (Cheapest First):")
by_price = sort_by_price(orders)
for o in by_price:
    print(o)

print("\nSearch by Order ID (1005):",
      search_order_by_id(sorted(orders, key=lambda x: x.order_id), 1005))
print(f"Search by Restaurant (Sushi Express):", search_by_restaurant(orders, "Sushi Express"))

```

Output:

```

--- Online Food Delivery Platform ---

Original Orders:
Order(ID:1005, Pizza Palace, Time:35min, $18.99, out_for_delivery)
Order(ID:1001, Burger King, Time:25min, $12.50, preparing)
Order(ID:1008, Sushi Express, Time:40min, $22.99, pending)
Order(ID:1003, Taco Bell, Time:20min, $15.75, delivered)
Order(ID:1007, Chinese Wok, Time:30min, $16.99, out_for_delivery)
Order(ID:1002, Subway, Time:15min, $9.99, delivered)
Order(ID:1006, Dominos, Time:28min, $25.50, preparing)

Sorted by Delivery Time (Fastest First):
Order(ID:1002, Subway, Time:15min, $9.99, delivered)
Order(ID:1003, Taco Bell, Time:20min, $15.75, delivered)
Order(ID:1001, Burger King, Time:25min, $12.50, preparing)
Order(ID:1006, Dominos, Time:28min, $25.50, preparing)
Order(ID:1007, Chinese Wok, Time:30min, $16.99, out_for_delivery)
Order(ID:1005, Pizza Palace, Time:35min, $18.99, out_for_delivery)
Order(ID:1008, Sushi Express, Time:40min, $22.99, pending)

Sorted by Price (Cheapest First):
Order(ID:1002, Subway, Time:15min, $9.99, delivered)
Order(ID:1001, Burger King, Time:25min, $12.50, preparing)
Order(ID:1003, Taco Bell, Time:20min, $15.75, delivered)
Order(ID:1007, Chinese Wok, Time:30min, $16.99, out_for_delivery)
Order(ID:1005, Pizza Palace, Time:35min, $18.99, out_for_delivery)
Order(ID:1008, Sushi Express, Time:40min, $22.99, pending)
Order(ID:1006, Dominos, Time:28min, $25.50, preparing)

Search by Order ID (1005): Order(ID:1005, Pizza Palace, Time:35min, $18.99, out_for_delivery)
Search by Restaurant (Sushi Express): Order(ID:1008, Sushi Express, Time:40min, $22.99, pending)

```

Explanation: This system manages thousands of food orders.

Binary Search helps quickly locate orders by ID.

Merge Sort is used to sort orders by delivery time or price.

